Overnight work, 2026-10-12 → 13

Context: you asked for everything possible to be done unattended, then cleanup.

The product now exists and works

```
pip install -e . && cetp selftest      # PASSES
```

check	result
test suite	**28/28 pass**
`cetp selftest`	PASSES (redaction, prediction, regression, calibration)
gate demo	all **4** verdict branches fire
canonical headline	analytical **21.61%** vs baseline **23.05%**, CI [0.32, 2.56]
Docker artifact	**reproduces the headline exactly**
workload	22 TPC-H + 99 TPC-DS + 113 JOB, all validating

Blockers closed

1. Shipped models were stale three ways — FIXED. Retrained on the buffer-corrected dataset, pickled with the pinned scikit-learn, and switched to the **multi-core** compute signal the headline requires. `gate/cetp.yml` and the demo scenarios were moved to the same scale, because a gate whose config disagrees with its models is worse than a merely stale one. Five locked prediction values and the four demo scenarios changed as a direct consequence; **each was reviewed against its predecessor before being updated** (note above `KNOWN_ANSWER_TRIPLES` in `tests/test_e2e.py`). Previous artifacts are in `models_backup/`. Remaining caveat, documented not hidden: the compute axis is probed at `nproc` rather than the achievable DOP, which overstates usable capacity for the two 4-vCPU machines.

2. New signature wired in — FIXED. `cetp.signature` resolves machines from `models/hardware_signatures.json` (probe v2, seven axes, probed at the achievable DOP) or from the five frozen legacy machines, and **refuses to mix the two scales** because the bandwidth probes differ. `cetp signature probe` onboards a new machine in ~40 seconds.

3. Plan redaction — DONE. `cetp/redact.py`, two levels, with `assert_clean()` that re-scans output and raises if any literal survives. Verified on a real plan and a PII torture case (emails, SSNs, numeric ids stripped; structure preserved).

4. Packaging — DONE. `pyproject.toml` with `cetp` and `cetp-redact` entry points. **Runtime dependencies are empty** — the shipped predictor uses the analytical formula plus `models/calibration.json`, not a pickle, so it cannot go stale and is immune to scikit-learn version coupling. Plus `.github/workflows/cetp-regression-check.yml`.

5 & 6. Accuracy and guarantees — addressed by reframing, not by pretending. `cetp regress` is the documented default: comparing one query before/after a change cancels the hardware term (`hardware_term` reports how much did not, and `trust` grades the answer). Absolute mode prints its

own expected error and every applicable warning.

7. VS Code compute score — FIXED. New `cetp.measureMachine` command runs the probe and saves a profile, replacing a text field no human could fill. TypeScript compiles clean.

8. Experiments overwriting releases — FIXED. The analysis scripts write to `models_candidate/`; `make promote-models` backs up, promotes, and **fails if the locked tests break**.

The research result that changed

Item 3 (causal plan drift) first produced a fabricated number, and I caught it. v1 reported a 1.54× "plan effect" over cells where the plan was *identical* — impossible. Two silent bugs: `pg_hint_plan` was never `LOAD`ed into the session (so no plan was ever actually forced, and verification passed only where there was nothing to force — 100% of identical-plan cells, 0% of the ones that mattered), and the paired arms were measured in separate sweeps so cache state drifted (q20: 13,304 ms vs 539 ms for the same plan).

Both fixed, and the corrected experiment **reverses Tier 3's conclusion**: when the plan actually changes, plan divergence accounts for **70.5%** of the runtime difference (pure plan effect 1.673× vs 1.145× for settings, over 30 verified cells). Tier 3's ~16% was diluted by averaging over pairs where the plan never changed. **N1 is now the strongest finding in the project**, with one real caveat: forcing verified on only 41% of differing-plan cells, and the failures are not random.

Also done: **item 2** (bandwidth probe replaced; registry-backed; your M1 measures 34.90 GFLOPS at DOP vs 58.84 at `nproc` — the 1.69× bug, confirmed), **item 5** (version transfer: real effect, 0.708×–1.413×, but 4 versions cannot model it), **item 6** (Docker artifact, verified).

Item 1 (container grid) was written but not run. It needs ~90 minutes of exclusive machine time and I spent that on the product blockers instead, which is what you asked for. Run it with `python3 scripts/collection/container_grid.py --setup` then `--collect`.

Cleanup

Removed, all regenerable or superseded: `models_candidate/`, `.docker-noauth/`, `cetp.egg-info/`, `vscode-extension/out/`, `__pycache__`, `*.pyc`, `.DS_Store`, pg_hint_plan `.o` objects, and the old `dist/` (repackaged fresh).

Kept deliberately: `models_backup/` (your rollback), `scripts/collection/tpch_data/` (1.0 GB, needed to reload any PostgreSQL version or container), `data/external/` (tpcds-kit, JOB queries, pg_hint_plan source), `vscode-extension/node_modules/`, `results/plan_drift_causal_raw.csv` (the record of the failed v1), and `data/raw_buggy_do_not_use/` (buffer-bug provenance). Nothing large was tracked by git.

Nothing is committed

~75 files changed or added. You have never asked me to commit, so I have not.